

Backend Engineer (5 YOE)

Critical Interview Q&A

Big tech (Meta / Google / Amazon style) - verbatim answers

1. Database Indexing

Q: How do you decide the column order in a composite index?

Model answer (speak this):

The rule is Equality, Range, Sort. Columns used in equality filters come first, columns used in range filters or ORDER BY come last. The most selective equality column should be leftmost so the B-tree narrows the result set quickly.

For example, given WHERE gym_id = ? AND status = 'success' AND created_at BETWEEN ? AND ? ORDER BY created_at DESC, the correct index is (gym_id, status, created_at DESC). This lets the database use the index for filtering, range scanning, and the sort order in one pass.

Q: What is a partial index and when would you use it?

Model answer (speak this):

A partial index only indexes rows that match a WHERE clause. It is smaller, fits in memory better, and is faster to maintain. I use it when queries always filter on the same value AND that value is a small percentage of total rows.

If status = 'success' is 5 percent of rows, a partial index on (gym_id, created_at) WHERE status = 'success' is much smaller. If success is 95 percent of rows, a partial index gives no benefit because it covers almost the whole table - a regular composite index is fine.

2. Python Concurrency and FastAPI

Q: What is the GIL and how does it affect threading?

Model answer (speak this):

The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on a multi-core machine. So for CPU-bound work in pure Python, threading does not give parallelism - only one core works at a time.

The GIL is released during I/O operations and during certain C extension calls. That is why threading is still useful for I/O-bound work like network calls or DB queries. For true CPU parallelism, use multiprocessing - each process has its own interpreter and its own GIL.

Q: When do you use threading vs multiprocessing vs asyncio?

Model answer (speak this):

Threading for I/O-bound work at small scale or in sync codebases. Multiprocessing for CPU-bound work where I need real parallelism across cores. Asyncio for I/O-bound work at large scale - one event loop can handle thousands of concurrent connections with much less overhead than threads.

Q: Why is asyncio.gather on a single SQLAlchemy async session unsafe?

Model answer (speak this):

An AsyncSession is not concurrency-safe. Internally one session maps to one database connection and one transaction. When two coroutines run gather on the same session, both try to use that single connection simultaneously and you get InvalidRequestError or greenlet spawn errors.

The fix is to give each parallel coroutine its own session from a session factory. Each session pulls its own connection from the pool, so concurrent queries are safe.

Q: FastAPI: difference between def and async def routes?

Model answer (speak this):

An async def route runs on the event loop. While it awaits I/O, the loop handles other requests. A regular def route is run by FastAPI in a thread pool so it does not block the event loop, but each request occupies a thread for its full duration.

The dangerous case is using a sync blocking call inside an async route - that blocks the event loop and freezes all other requests. The rule: async def goes with await on an async library; sync def goes with sync libraries.

3. Concurrency, Race Conditions, ACID

Q: Walk me through the lost update race condition.

Model answer (speak this):

Two requests read the same row at the same time, both see balance 100. Both check that 100 is enough, both compute new_balance = 20 in memory, both write UPDATE balance = 20. The second UPDATE blindly overwrites the first. The balance dropped by 80 but two debits happened - the gym effectively got 80 credits free. This is the classic read-modify-write race.

Q: How do you fix it?

Model answer (speak this):

Four common approaches. First and best for arithmetic: atomic UPDATE with the guard in the WHERE clause - UPDATE gym_credits SET balance = balance - 80 WHERE gym_id = ? AND balance >= 80, then check rows affected. The DB does the read, check, and write as a single atomic operation.

Second: SELECT FOR UPDATE pessimistic lock for complex logic that needs to happen between read and write. Third: optimistic locking with a version column for low-conflict high-concurrency cases. Fourth: a distributed Redis lock when the critical section spans multiple databases or services.

Q: Explain ACID in plain English.

Model answer (speak this):

Atomicity: a transaction is all or nothing - if any step fails the whole transaction rolls back. Consistency: every transaction moves the database from one valid state to another, satisfying all constraints. Isolation: concurrent transactions do not see each other's uncommitted changes - they appear to run serially. Durability: once committed, the change survives crashes because it was written to the write-ahead log on disk before commit returned.

Q: Isolation levels - which DB uses which by default?

Model answer (speak this):

READ UNCOMMITTED allows dirty reads - rarely used. READ COMMITTED prevents dirty reads but allows non-repeatable reads - this is the PostgreSQL default. REPEATABLE READ prevents dirty and non-repeatable reads - this is the MySQL InnoDB default, and InnoDB also prevents phantom reads using

next-key locks. SERIALIZABLE prevents all three anomalies but is the slowest because it essentially serializes conflicting transactions.

4. Authentication and Authorization

Q: How do you design JWT authentication for a web and mobile app?

Model answer (speak this):

I use a two-token pattern: a short-lived access token, around 15 minutes, and a long-lived refresh token, around 30 days. On web the refresh token sits in an httpOnly, Secure, SameSite cookie - JavaScript cannot read it, so XSS cannot steal it. The access token lives in memory. On mobile, both tokens go into the secure Keychain or Keystore.

I never store JWTs in localStorage on the web - it is vulnerable to XSS. Logout invalidates the refresh token in the database and pushes the access token's JTI to a Redis blocklist until it expires.

Q: JWTs are stateless - how do you actually revoke them on logout or account suspension?

Model answer (speak this):

Three options. First, keep access tokens short - 5 to 15 minutes - and check the user's status in the database on every refresh. Even if the access token is still valid, the user cannot get a new one after suspension.

Second, a Redis blocklist of revoked token IDs with TTL equal to the remaining token lifetime - every request checks Redis. Third, a token_version claim in the JWT compared against a version column in the user row - on suspend, bump the version and all existing tokens become invalid.

Q: Difference between 401 and 403?

Model answer (speak this):

401 Unauthorized means the request is not authenticated - no token, expired token, or invalid signature. The fix is to log in or refresh. 403 Forbidden means the request is authenticated but not allowed - we know who you are, but you do not have permission for this resource.

Example: a trainer at Gym A trying to delete a member at Gym B gets 403. A user sending no token at all gets 401.

5. Caching and Redis

Q: How would you build a real-time leaderboard for 50K members?

Model answer (speak this):

I use a Redis sorted set, ZSET. The member is the user ID, the score is calories burned. ZINCRBY updates the score in $O(\log N)$ on every workout. ZREVRANGE 0 99 returns the top 100 in $O(\log N + 100)$. The DB is the source of truth - I run a periodic rebuild from MySQL to self-heal any drift.

Q: What is a cache stampede and how do you prevent it?

Model answer (speak this):

Also called thundering herd. When a popular cache key expires, many requests see a miss at the same time and all hit the database concurrently. The DB gets hammered and can crash.

Fixes: a single-flight lock so only one request rebuilds the cache while others wait, stale-while-revalidate so requests serve stale data and trigger a background refresh, probabilistic early expiration so requests refresh

slightly before TTL, or for predictable values like a daily 'member of the day', a cron job that pre-warms the cache before the old one expires.

Q: Cache-aside vs write-through vs write-behind?

Model answer (speak this):

Cache-aside: app reads cache, falls back to DB on miss, populates the cache. Most common. Write-through: writes go to cache and DB in the same request - consistent but slightly slower. Write-behind: writes go to cache, then a background job persists to DB - fastest writes, eventual consistency.

6. HTTP and REST API Design

Q: PUT vs PATCH?

Model answer (speak this):

PUT replaces the entire resource - the body must contain the full representation, and missing fields are treated as unset. PATCH applies a partial update - the body contains only the fields to change. PATCH is more common in modern APIs because it is more forgiving.

Q: What is idempotency and which HTTP methods are idempotent?

Model answer (speak this):

An operation is idempotent if performing it once has the same end-state as performing it N times. GET, HEAD, OPTIONS, PUT, and DELETE are idempotent. POST is not. PATCH usually is.

It matters because network timeouts are ambiguous - the client does not know if the request reached the server. Safe retries require idempotency. For non-idempotent POSTs like creating a payment, the client sends an Idempotency-Key header with a UUID. The server caches the response for that key and returns it on retry. Stripe, Razorpay, AWS all use this pattern.

Q: Key HTTP status codes you must use correctly.

Model answer (speak this):

201 Created for successful resource creation. 204 No Content for successful update with no body. 400 Bad Request for malformed input. 401 Unauthorized for missing or invalid auth. 403 Forbidden for authorized-but-not-allowed. 404 Not Found for missing resource. 409 Conflict for duplicate or stale-version writes. 422 Unprocessable Entity for semantically invalid input. 429 Too Many Requests for rate limits - include Retry-After header. 500 for unhandled server errors. 503 Service Unavailable for downstream outages.

7. Microservices and System Design

Q: Monolith or microservices? How do you decide?

Model answer (speak this):

Microservices solve organizational problems, not technical ones. With a small team and a clean modular monolith, you ship faster, debug easier, and pay no distributed-systems tax. I would split into services only when team size and deployment friction make coordinated releases painful - usually past 20 to 25 engineers across multiple product areas, or when scaling needs across subsystems diverge significantly.

My starting architecture is a modular monolith with clear internal module boundaries. When the time comes to split, each module already has a clean interface to extract.

Q: Sync REST vs event-driven for service-to-service communication?

Model answer (speak this):

I default to event-driven for internal communication. The producer publishes a domain event - `payment.succeeded` - to a broker like Kafka or SQS, and any service that cares subscribes. This decouples the producer from consumers, survives consumer outages because events queue up, and makes it trivial to add a new consumer next month without touching the producer.

Sync REST or gRPC only when the producer actually needs the consumer's response - for example, calling a fraud check before completing a payment.

Q: What is the outbox pattern?

Model answer (speak this):

It solves the problem where the DB commit succeeds but publishing the event to Kafka fails, leaving the system inconsistent. Instead of publishing directly, the producer writes the event to an outbox table in the same DB transaction as the business write. A separate process reads the outbox and publishes to Kafka. Now the write and the event are atomic - either both happen or neither does.

Q: Explain the CAP theorem.

Model answer (speak this):

In a distributed system, when a network partition happens, you have to choose between consistency and availability. You cannot have both during a partition. Since partitions are inevitable in any real distributed system, the real choice is CP or AP.

CP systems refuse to serve rather than return stale data - Postgres with synchronous replication, etcd, ZooKeeper. AP systems stay up and return possibly-stale data - Cassandra, DynamoDB eventually-consistent reads, DNS. Modern databases often make this tunable per query.

Q: Eight services, request times out somewhere - what do you need?

Model answer (speak this):

Two things: observability and resilience. For observability, the trinity is distributed tracing with OpenTelemetry so one `trace_id` follows the request through every service, structured logs that include the `trace_id`, and metrics using the RED method - rate, errors, duration - per service.

For resilience: timeouts on every RPC, circuit breakers that fail fast after repeated downstream failures, retries with exponential backoff and jitter, bulkhead isolation so one slow dependency does not exhaust shared thread pools, and graceful degradation so a non-critical service being down still lets the request succeed.

Q: Kafka vs RabbitMQ vs SQS - one-line each.

Model answer (speak this):

Kafka: high-throughput, durable, append-only log. Pick for event streaming, replay, and multiple consumer groups re-reading the same events.

RabbitMQ: classic broker with rich routing - topic, direct, fanout exchanges. Pick for task queues, RPC, and complex routing.

SQS: AWS-managed simple queue with zero ops. Pick when you are on AWS and want pure queue semantics without running infrastructure.

8. One-Liners You Must Recall Instantly

Composite index rule:	Equality, then range, then sort. Most selective column first.
Partial index:	Smaller index over a WHERE-filtered subset. Use when filter is on a rare value.
Lost update fix (best default):	Atomic UPDATE with the guard in the WHERE clause; check rows affected.
MySQL default isolation:	REPEATABLE READ (InnoDB also blocks phantoms via next-key locks).
Postgres default isolation:	READ COMMITTED.
401 vs 403:	401 = who are you. 403 = I know you, you can't do this.
Idempotent HTTP methods:	GET, HEAD, OPTIONS, PUT, DELETE. POST is not.
Idempotency for POST:	Client sends Idempotency-Key header; server caches response per key.
JWT logout problem:	JWTs are stateless. Use short access tokens + DB-tracked refresh + Redis blocklist.
Cache stampede fix:	Single-flight lock, stale-while-revalidate, or pre-warm before expiry.
Redis leaderboard:	Sorted set. ZINCRBY to update, ZREVRANGE to read top N.
GIL:	One thread runs Python bytecode at a time. Released during I/O. Multiprocessing for CPU work.
asyncio.gather + AsyncSession:	Unsafe. One session = one connection. Give each task its own session.
FastAPI sync inside async:	Blocks the event loop. Either go fully async or use a sync def route.
is vs ==:	is = same object in memory. == = equal value. Use is only for None, True, False.
PUT vs PATCH:	PUT replaces the whole resource. PATCH updates fields you send.
Modular monolith:	Default architecture until org size forces a split.
Outbox pattern:	Write event to DB outbox in same transaction; relay to Kafka after.
CAP theorem:	During partition, choose Consistency or Availability. Real systems are tunable.
Observability trinity:	Distributed tracing, structured logs, metrics. All linked by trace_id.
Resilience pattern set:	Timeouts, circuit breakers, retries with backoff + jitter, bulkheads.
Kafka vs SQS vs RabbitMQ:	Kafka = event log + replay. SQS = managed simple queue. RabbitMQ = rich routing.

How to use this: speak the model answers in your own rhythm but keep the key phrases - 'equality-range-sort', 'lost update', 'idempotency key', 'single-flight lock', 'modular monolith', 'outbox pattern', 'observability trinity'. Interviewers grade on whether you reach for the right vocabulary, not on word-for-word recital.